

Bible API Integration Guide

How BibleApp talks to bible.helloao.org — a walkthrough of every layer, and a reusable recipe for doing this again on your own.

This document explains, file by file, how the Bible-text network API is wired into BibleApp — from the raw HTTP request all the way to what appears on screen. It's written so you can read it top to bottom once to understand the current code, then come back to Section 6 whenever you need to bolt a *different* API onto this app (or any other Android app) in the future.

1. The API, in one paragraph

BibleApp gets Bible text from the free **Free Use Bible API** at bible.helloao.org. It's a plain REST API that returns JSON — no auth key, no login. The app only calls two endpoints out of everything the API offers:

- GET /api/available_translations.json — the full catalog of ~1,200 translations, fetched once to populate the version picker.
- GET /api/{translationId}/complete.json — the **entire** Bible for one translation (every book, chapter, verse) in a single 5–10 MB response.

DESIGN DECISION

The API also has a books-only endpoint and a per-chapter endpoint (one request per chapter, ~1,189 requests for a whole Bible). BibleApp deliberately uses only the single “complete.json” request instead — one big download beats a thousand small ones, and it lets the app work fully offline afterward.

2. Architecture at a glance

The integration is split into clear layers, each with one job. Data flows straight down; state updates flow back up automatically through Kotlin StateFlow. This separation is the single most important idea in this document — Section 6 turns it into a checklist you can reuse.

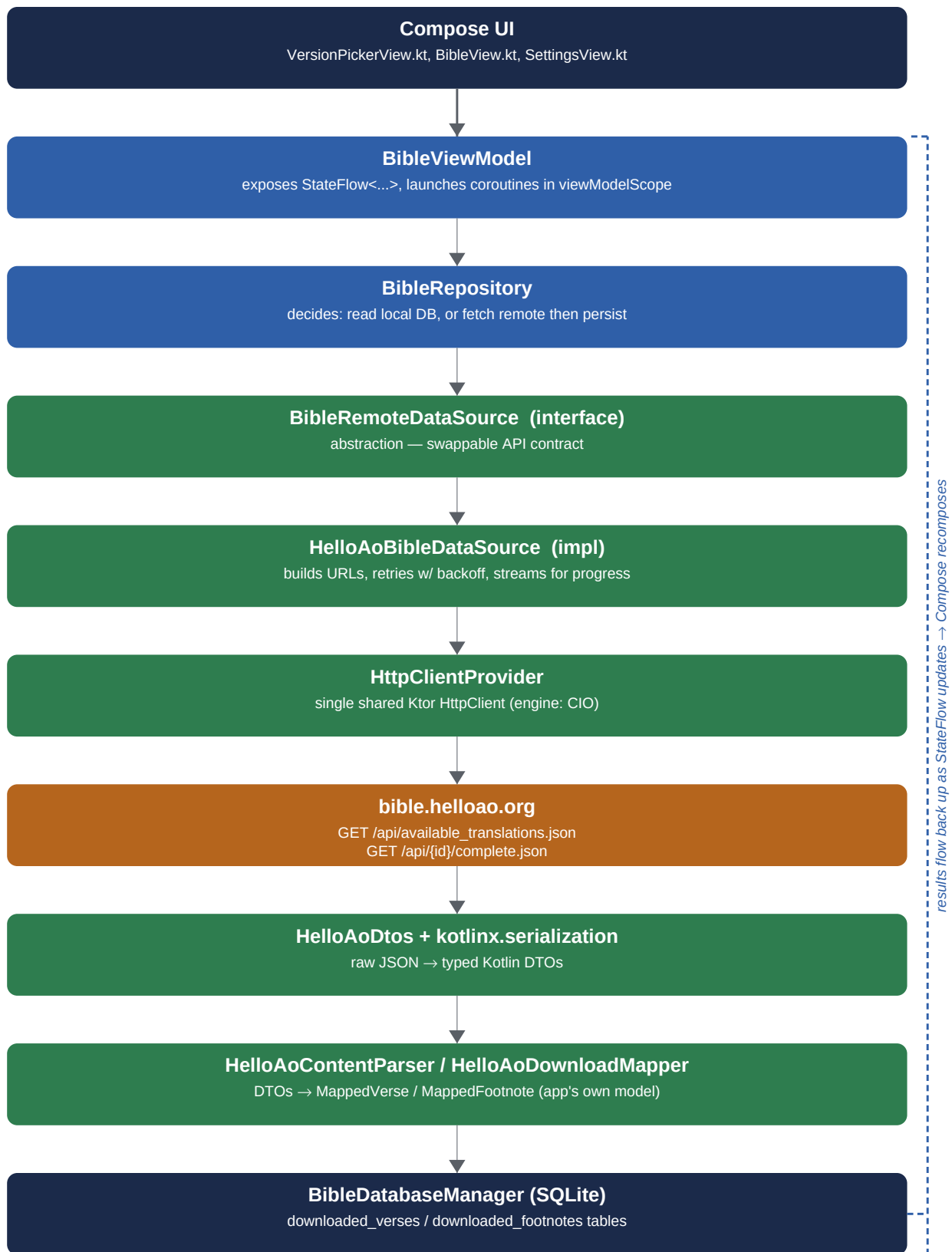


Figure 1 — request/response flow from tap to screen.

3. Step-by-step walkthrough of the code

This is the order the pieces were actually built in — and the order you'd rebuild them in for a new API. Each step names the real file in the project.

3.1 — Add the networking libraries

`app/build.gradle.kts`

Gradle dependencies

Two libraries do all the heavy lifting: **Ktor** (an HTTP client built for Kotlin coroutines) and **kotlinx.serialization** (turns JSON into Kotlin data classes and back).

```
implementation("io.ktor:ktor-client-core:2.3.8")
implementation("io.ktor:ktor-client-cio:2.3.8")           // CIO = the actual engine that sends bytes
implementation("io.ktor:ktor-client-content-negotiation:2.3.8")
implementation("io.ktor:ktor-serialization-kotlinx-json:2.3.8")
implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.6.3")
```

And in `AndroidManifest.xml`, the app declares it needs network access at all:

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
```

COMMON PITFALL

Without the `INTERNET` permission every network call fails at the OS level, not the app level — this is the single most common “why does my API call silently fail” bug for beginners on Android.

3.2 — One shared HTTP client for the whole app

`data/remote/HttpClientProvider.kt`

singleton Ktor client

Creating an `HttpClient` is expensive (it sets up connection pools, thread pools, etc.), so the app builds exactly one and reuses it everywhere via a Kotlin object (Kotlin's built-in singleton).

```
object HttpClientProvider {
    val json = Json {
        ignoreUnknownKeys = true    // API adds a field you don't model yet? App won't crash.
        isLenient = true
    }

    val client: HttpClient by lazy { // "by lazy" = built only on first use, then cached
        HttpClient(CIO) {
            install(ContentNegotiation) { json(json) } // auto encode/decode JSON <-> Kotlin objects
            install(HttpTimeout) {
                requestTimeoutMillis = 15_000
                connectTimeoutMillis = 10_000
                socketTimeoutMillis = 15_000
            }
        }
    }
}
```

KOTLIN CONCEPT

object instead of **class** guarantees only one instance ever exists (no “new HttpClient()” scattered around). **by lazy** means it isn't created at app startup — only the first time something actually needs it, and every caller after that gets the same cached instance.

3.3 — Describe the JSON shape as Kotlin data classes (DTOs)

`data/remote/HelloAoDtos.kt`

Data Transfer Objects

A **DTO** (Data Transfer Object) is a data class whose only job is to mirror the exact shape of the JSON a server sends. The `@Serializable` annotation lets `kotlinx.serialization` generate the JSON ↔ Kotlin conversion code automatically — no manual parsing.

```
@Serializable
data class HelloAoTranslationDto(
    val id: String,
    val name: String,
    val englishName: String,
    val shortName: String? = null,    // "= null" -> optional; missing key won't crash decoding
    val language: String,
    val textDirection: String? = null
)
```

Field names must match the JSON keys (or be told how to via annotations). Fields the app doesn't care about simply aren't declared — combined with `ignoreUnknownKeys` from step 3.2, the API can add new fields at any time without breaking the app.

One tricky part of this particular API: a chapter's `content [ ]` array mixes several different shapes in one list (headings, line breaks, verses). Rather than fight that into a rigid Kotlin type, it's kept as raw, untyped JSON and walked by hand later (Section 3.6):

```
@Serializable
data class HelloAoChapterContentDto(
    val number: Int,
    val content: List<JsonElement> = emptyList(),    // deliberately untyped/heterogeneous
    val footnotes: List<HelloAoFootnoteDto> = emptyList()
)
```

3.4 — Define an abstraction before the implementation

`data/remote/BibleRemoteDataSource.kt`

interface / contract

Before writing any networking code, the app declares *what* a Bible data source can do, without saying *how*. Everything above this layer (repository, ViewModel, UI) only ever depends on this interface — never on Ktor or `helloao.org` directly.

```
interface BibleRemoteDataSource {
    suspend fun getAvailableTranslations(): List<BibleTranslation>

    suspend fun downloadTranslation(
        translationId: String,
        onProgress: (Float) -> Unit = {}
    ): DownloadedTranslation
}
```

WHY THIS MATTERS

This is **dependency inversion**: high-level code (ViewModel, Repository) depends on an interface, and the low-level detail (“call this specific API with Ktor”) depends on that same interface too, instead of the high-level code depending on the low-level detail directly. Practical payoff: swapping bible.helloao.org for a different Bible API later means writing one new class that implements this interface — zero changes anywhere else. It also makes the ViewModel trivially testable with a fake implementation, no real network needed.

3.5 — Implement the real network calls

data/remote/HelloAoBibleDataSource.kt concrete Ktor client

The simple call — fetch the translation catalog:

```
override suspend fun getAvailableTranslations(): List<BibleTranslation> {
    val response = client.get("$baseUrl/api/available_translations.json")
    if (response.status != HttpStatusCode.OK) {
        throw IOException("Failed to load translations: server returned ${response.status}")
    }
    val dto: HelloAoTranslationsResponseDto = response.body() // Ktor auto-decodes JSON -> DTO here
    return dto.translations.map { it.toBibleTranslation() } // DTO -> app's own domain model
}
```

The harder call — downloading a whole 5–10 MB translation — adds two things a simple GET doesn't need: **retry with backoff** for flaky connections, and **manual streaming** so the UI can show a real progress bar instead of freezing until the whole file arrives.

```
for (attempt in 1..MAX_FETCH_ATTEMPTS) {
    try {
        val dto = fetchComplete(url, onProgress)
        return DownloadedTranslation(/* ...mapped fields... */)
    } catch (e: CancellationException) {
        throw e // coroutine was cancelled -> always rethrow, never swallow
    } catch (e: TranslationNotFoundException) {
        throw e // 404 is permanent -> retrying can't help, fail fast
    } catch (e: Exception) {
        if (!isRetriableNetworkError(e)) throw e
        val backoffMs = RETRY_BASE_DELAY_MS * (1L shl (attempt - 1)) // 800ms, 1600ms, 3200ms...
        delay(backoffMs)
    }
}
```

```

val channel: ByteReadChannel = response.bodyAsChannel() // read the body in chunks, not all at once
val chunk = ByteArray(32 * 1024)
while (!channel.isClosedForRead) {
    val read = channel.readAvailable(chunk)
    if (read == -1) break
    buffer.write(chunk, 0, read)
    totalRead += read
    // this drives the on-screen progress bar
    onProgress((totalRead.toFloat() / expectedBytes).coerceIn(0f, 0.95f))
}

```

WHY THIS MATTERS

Exponential backoff (800ms → 1.6s → 3.2s) gives a shaky connection breathing room to recover instead of hammering the server three times in a row. Distinguishing a 404 (permanent, don't retry) from a timeout (transient, retry) is what `NetworkErrorClassifier.kt` exists for — see 3.7.

3.6 — Parse the messy/polymorphic parts of the JSON by hand

data/remote/HelloAoContentParser.kt

manual JSON walker

Recall from 3.3 that a chapter's content `[]` mixes headings, line breaks, and verses in one array, and a verse's own content mixes plain strings with objects like `{"text": "...", "wordsOfJesus": true}`. Kotlin serialization can't map that onto one fixed type, so this file walks the raw `JsonElement` list item by item and builds up a clean internal structure (headings + a list of “runs” of styled text) as it goes.

```

when (type) {
    "heading"    -> pendingHeadings += StoredHeading(type = "heading", text = ...)
    "line_break" -> pendingParagraphBreak = true
    "verse"     -> {
        val runs = buildRuns(obj["content"]?.jsonArray ?: JsonArray(emptyList()))
        verses += ParsedVerse(number, plainText, StoredVerseContent(pendingHeadings, runs))
        pendingHeadings.clear()
    }
}

```

This file is a pure Kotlin object with no Android imports at all — which is a deliberate, testable design choice: it can be unit-tested on the plain JVM in milliseconds, no emulator required.

3.7 — Decide which failures are worth retrying

data/remote/NetworkErrorClassifier.kt

retry policy

```

fun isRetriableNetworkError(e: Throwable): Boolean = when (e) {
    is IOException                -> true // DNS blips, socket resets, read timeouts...
    is UnresolvedAddressException -> true // DNS failure (doesn't extend IOException on Android!)
    is HttpRequestTimeoutException -> true
    is ConnectTimeoutException    -> true
    is SocketTimeoutException     -> true
    else                          -> false // bad params/malformed body -> retry won't help
}

```

A tiny, single-purpose function like this pays for itself: the retry loop in 3.5 stays generic (“is this retrievable, yes/no”) and every rule about what counts as “transient” lives in exactly one place.

3.8 — Map API data onto the app's own model, then persist it

`data/remote/HelloAoDownloadMapper.kt` + `data/local/BibleDatabaseManager.kt` DTO -> domain -> SQLite

The API numbers/orders books its own way. The mapper translates that onto the app's internal 1–66 canonical book numbering (Genesis=1 ... Revelation=66) so nothing above this layer ever needs to know the source API's own conventions — or that helloao.org exists at all.

```
complete.books.forEach { book ->
    // e.g. apocrypha -> not supported by the local schema, skip
    if (book.order !in 1..66) { skippedBooks++; return@forEach }
    book.chapters.forEach { chapterEntry ->
        HelloAoContentParser.extractVerses(chapterEntry.chapter.content).forEach { v ->
            verses += MappedVerse(
                bookId = book.order, chapter = chapterEntry.chapter.number,
                verse = v.number, text = v.plainText, richContent = v.richContent
            )
        }
    }
}
```

The result is written to SQLite in one batched transaction, and only after the entire network response has been fully fetched and parsed in memory:

```
db.beginTransaction()
downloaded.verses.forEach { verse ->
    db.execSQL(
        "INSERT OR REPLACE INTO downloaded_verses (...) VALUES (?, ?, ?, ?, ?, ?)",
        arrayOf(...)
    )
}
db.setTransactionSuccessful()
db.endTransaction()
```

DESIGN DECISION

This is the app's **offline-first** strategy: a translation is downloaded once, fully, into SQLite. Every screen after that reads from the local database — never the network.

`BibleRepository.getChapter()` (next section) never touches `BibleRemoteDataSource` at all; only the one-time download path does.

REAL BUG THIS CODE AVOIDS

The comment in the real source file explains a real bug that was hit and fixed: Android's SQLite transactions are tied to whichever thread calls `beginTransaction()`, but a suspended coroutine can resume on a *different* pool thread. Awaiting a network call in the middle of an open transaction intermittently crashed with “no current transaction.” The fix: do 100% of the network I/O and parsing *before* opening the transaction, so everything inside `beginTransaction()/endTransaction()` is synchronous with no suspension points.

3.9 — The Repository: one place that decides where data comes from

data/repository/BibleRepository.kt

mediator

The ViewModel never talks to the network or the database directly. It only talks to `BibleRepository`, which owns both a `BibleRemoteDataSource` and `BibleDatabaseManager`, and decides which one to use for a given call.

```
class BibleRepository(private val context: Context, private val remote: BibleRemoteDataSource) {

    suspend fun getChapter(bookId: Int, chapter: Int, versionId: String): List<VerseUI> =
        withContext(Dispatchers.IO) { // <-- always local, never network
            BibleDatabaseManager
                .getVersesByChapter(context, bookId, chapter, versionId)
                .map { it.toUI() }
        }

    // <-- this one call is the only network hit in the whole class
    suspend fun getAllVersions(): List<BibleTranslation> =
        remote.getAvailableTranslations()

    suspend fun downloadVersion(translationId: String, onProgress: (Float) -> Unit = {}) =
        BibleDatabaseManager.downloadAndSaveVersion(context, translationId, remote, onProgress)
}
```

This is the classic **Repository pattern**: one seam in the whole app where “where does this data actually live” is decided. Add a cache, change the backing API, add a second data source — this is the only file that would need to change.

3.10 — ViewModel: coroutines in, StateFlow out

viewModel/BibleViewModel.kt

UI state holder

The ViewModel starts coroutines with `viewModelScope.launch { }` (auto-cancelled if the screen is destroyed), calls the suspend functions on the repository, and republishes the result as `StateFlow` — observable state the Compose UI collects and redraws on.

```
private suspend fun downloadAndSwitch(versionId: String): Boolean {
    _downloadingVersionId.value = versionId
    _downloadProgress.value = 0f
    _downloadError.value = null

    val result = repository.downloadVersion(
        translationId = versionId,
        onProgress = { _downloadProgress.value = it } // Ktor bytes -> StateFlow -> progress bar
    )

    return result.fold(
        onSuccess = { switchToVersion(versionId); true },
        onFailure = { _downloadError.value = it.message ?: "Download failed"; false }
    )
}
```

Notice `Result<VersionDownloadSummary>` as the return type all the way from `BibleDatabaseManager` up through the repository to here. Kotlin's built-in `Result` type carries either a

success value or an exception, and `.fold(onSuccess, onFailure)` forces both cases to be handled explicitly — there's no way to silently forget the error path.

3.11 — Compose UI: just collects state and calls functions

screens/VersionPickerView.kt

Compose screen

By the time data reaches the UI layer, it's plain state — lists, floats, nullable strings. The screen never knows an HTTP request or a SQL query happened at all; it just reads `StateFlow` values via `collectAsState()` and calls `ViewModel` functions on tap: `viewModel.selectVersion(id)`, `viewModel.loadAvailableVersions()`. This is the payoff of every layer above — the UI code stays simple because all the complexity (retries, threading, parsing, storage) was handled before it ever got here.

4. Concepts worth understanding deeply

These ideas aren't specific to Bible data or to this app — they're the reusable core of *any* API integration on Android/Kotlin.

suspend fun	A function that can pause without blocking the thread while it waits (e.g. for network I/O), then resume later. Callable only from a coroutine or another suspend function.
Coroutine / viewModelScope	A lightweight, cancellable unit of async work. <code>viewModelScope.launch { }</code> ties the coroutine's lifetime to the ViewModel, so it's auto-cancelled if the user navigates away mid-request.
Dispatchers.IO	Tells a coroutine to run on a thread pool meant for blocking I/O (network, disk) instead of the main/UI thread, which must stay free to render frames.
StateFlow	An observable holder of the “latest value” that Compose UI collects and automatically redraws from whenever it changes — the reactive glue between ViewModel and UI.
DTO vs. domain model	A DTO (<code>HelloAoTranslationDto</code>) mirrors the server's JSON exactly. A domain model (<code>BibleTranslation</code>) is the app's own shape. Keeping them separate means an API's quirky field names never leak past the data/remote layer.
Interface / dependency inversion	High-level code (<code>Repository</code> , <code>ViewModel</code>) depends on an interface (<code>BibleRemoteDataSource</code>), not a concrete class. Swapping implementations — or faking one for tests — requires touching nothing else.
Result	Kotlin's built-in wrapper for “success value OR exception,” returned instead of thrown, so callers are nudged to handle both outcomes via <code>.fold(...)</code> .
Exponential backoff	Waiting progressively longer between retries (800ms, 1.6s, 3.2s...) instead of retrying instantly, to give transient failures time to clear.
Offline-first	Fetch once, store locally, read from local storage from then on. The app should be fully usable with no network after the initial download.

5. File map — quick reference

<code>HttpClientProvider.kt</code>	Singleton Ktor HttpClient + shared JSON config
<code>HelloAoDtos.kt</code>	@Serializable data classes mirroring the API's JSON
<code>BibleRemoteDataSource.kt</code>	Interface: the contract any Bible data source must satisfy
<code>HelloAoBibleDataSource.kt</code>	Concrete implementation: real GET requests, retries, progress streaming
<code>HelloAoContentParser.kt</code>	Hand-walks the polymorphic content[] JSON array into clean verse/heading data
<code>HelloAoDownloadMapper.kt</code>	Maps parsed API data onto the app's internal 1–66 book numbering
<code>NetworkErrorClassifier.kt</code>	isRetriableNetworkError() — transient vs. permanent failure
<code>BibleDatabaseManager.kt</code>	Persists downloaded verses/footnotes to SQLite; reads them back for the UI
<code>BibleRepository.kt</code>	Mediator: decides local DB vs. remote API for every call the ViewModel makes
<code>BibleViewModel.kt</code>	Orchestrates coroutines, exposes StateFlow to the UI, holds download progress/errors
<code>VersionPickerView.kt</code> / <code>BibleView.kt</code>	Compose screens — collect StateFlow, call ViewModel functions

6. Do it yourself: integrating a new API, step by step

Follow this checklist for any future API integration — a different Bible API, a weather API, anything that returns JSON over HTTP. It's exactly the pattern from Section 3, generalized.

Step 1 — Read the API's actual responses first

Before writing a single Kotlin file, call the real endpoints (browser, curl, or Postman) and look at the real JSON. Don't trust docs alone — the helloao DTOs in this app were “verified against the live API, not assumed from documentation.” Note: what's optional vs. required, what's inconsistently shaped (like `content[]`), and roughly how large a response is.

Step 2 — Add dependencies + permissions

- Ktor client libs (core, an engine like CIO, content-negotiation, kotlinx-json) in `build.gradle.kts`.
- `INTERNET` (and usually `ACCESS_NETWORK_STATE`) in `AndroidManifest.xml`.

Step 3 — Build (or reuse) a shared `HttpClient`

One client per app, as a lazy singleton object, with sane timeouts. If you're adding a *second* API to this same app, you can likely reuse the existing `HttpClientProvider.client` instead of building another.

Step 4 — Write DTOs that mirror the JSON exactly

- One `@Serializable` data class per JSON object shape.
- Mark anything not guaranteed present as nullable with a default.
- Keep `ignoreUnknownKeys = true` so new server fields don't crash old app versions.
- For any field that's a JSON array/object mixing incompatible shapes, keep it as raw `JsonElement` and parse it by hand (Step 7) rather than forcing an awkward sealed-class hierarchy.

Step 5 — Design the interface before the implementation

Ask: “what does the rest of the app actually need from this data source?”, and write that as an interface with suspend functions returning your *own* domain types — never the raw DTOs. This is the step that's easiest to skip and most valuable to keep.

Step 6 — Implement the calls

- `client.get(url)` for reads; check `response.status` before trusting `response.body()`.
- Wrap flaky/large calls in a retry loop that checks an `isRetriableNetworkError()`-style classifier.
- For big downloads, stream via `bodyAsChannel()` and report progress instead of one blocking `body()` call.
- Throw specific exceptions for permanent failures (404 -> a `NotFoundException`) so retry logic can distinguish them from transient ones.

Step 7 — Parse anything irregular, map to your domain model

Keep this pure Kotlin (no Android imports) wherever possible — it makes the trickiest logic unit-testable on the plain JVM, no emulator needed.

Step 8 — Decide: cache locally, or always hit the network?

If the data is large and mostly static (like Bible text), download once and store it (Room/SQLite), then read locally forever after. If it's small and changes often (like live weather), skip local storage and just call the network each time, perhaps with a short in-memory cache.

Step 9 — Put a Repository between the ViewModel and everything else

The ViewModel should only ever call `repository.something()`. The repository is the only class that knows whether “something” came from SQLite, from the network, or from both.

Step 10 — ViewModel: launch, await, publish

```
viewModelScope.launch {
    _isLoading.value = true
    val result = repository.fetchSomething()
    result.fold(
        onSuccess = { _data.value = it },
        onFailure = { _error.value = it.message }
    )
    _isLoading.value = false
}
```

Step 11 — UI just observes

`collectAsState()` the `StateFlow` values, render loading/error/data states, call ViewModel functions on user actions. No networking types (`HttpClient`, `DTOs`) should ever be imported in a Compose screen file.

RULE OF THUMB

If you get stuck deciding where a piece of new code belongs, ask: “does this know about JSON/HTTP?” → `data/remote`. “Does this know about SQL?” → `data/local`. “Does this decide local-vs-network?” → `repository`. “Does this hold UI state or call suspend functions?” → `ViewModel`. “Does this only render/observe?” → `Compose screen`. If code seems to belong in two of these at once, that's usually a sign it should be split.

7. Glossary

API	Application Programming Interface — here, a set of URLs a server exposes that return data (JSON).
REST	A common style of web API where each URL represents a resource and HTTP verbs (GET/POST/...) act on it.
JSON	JavaScript Object Notation — the text format almost all modern APIs use to send structured data.
DTO	Data Transfer Object — a Kotlin class shaped to match a server's JSON exactly.
Serialization	Converting a Kotlin object to JSON text (encode) or JSON text to a Kotlin object (decode).
Ktor	A Kotlin-first HTTP client/server library, built around coroutines.
CIO	"Coroutine I/O" — one of Ktor's pluggable network engines, pure Kotlin.
Coroutine	Kotlin's lightweight concurrency primitive; lets async code read top-to-bottom instead of nested callbacks.
suspend	Keyword marking a function that can pause/resume without blocking a thread.
StateFlow	An observable value holder from <code>kotlinx.coroutines</code> that Compose can watch and react to.
ViewModel	An Android Architecture Component that holds UI state and survives configuration changes (e.g. rotation).
Repository pattern	A single class that mediates between a ViewModel and one or more data sources (network, DB).
Dependency inversion	High-level code depends on an interface, not a concrete implementation, so the implementation can change or be swapped freely.
Offline-first	An app design where data is fetched once and cached locally, so the app keeps working without a network connection.
Exponential backoff	A retry strategy where the wait between attempts grows (e.g. doubles) each time.

Generated for BibleApp — covers the state of the data/remote, data/repository, and viewmodel layers as of the UI_Redesign branch.